

0.90.3. La fonction `defiler()` — faire glisser le fond

`#include <defiler>` : `defiler(x, y)` décale tout le fond de `x` pixels vers la gauche et `y` vers le haut.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `defiler(x, y)` fait **glisser tout le fond** : l'écran montre la carte à partir du pixel `(x, y)`. La carte fait 256 pixels de côté et **revient toute seule** au début.

La ligne à écrire : `#include <defiler>`.

Ses arguments : le décalage en pixels, en `x` puis en `y`.

Ce qu'elle coûte : environ **75 octets**, avec la mise en place de l'écran. L'appel seul : quatre octets.

Essaie : fais grandir `y` au lieu de `x` : le mot monte.

On retrouve `defiler` dans 10 leçons

[0.90.4](#), [0.95.1](#), [52](#), [61](#), [84](#), [85](#), [124](#), [125](#), [130](#), [137](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **331 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	52 o	← la console, à l'allumage — la mise en route de la console
main()	38 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	67 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	80 o	← CopierTuiles — 5 lettres sur 43
"GLISSE"	6 o	← texte() — un texte gravé

Lettres gravées : EGILS

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

texte()	× 1	11 o sur place, dans main() + EcrireTexte (10 o)
defiler()	× 1	7 o sur place, dans main()
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <defiler> : faire glisser le fond ----  
// À chaque image, x grandit d'un pixel : tout le fond glisse vers la gauche,  
// jusqu'à 60 pixels (7 cases et demie), puis s'arrête.  
  
#include <texte>  
#include <defiler> // defiler() : décaler tout le fond
```

2. UNE VARIABLE — X

```
uint8_t x = 0;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    texte(8, 8, "GLISSE");  
  
    while (true) {  
        image();  
        if (x < 60) {  
            x = x + 1; // un pixel de plus  
        }  
        defiler(x, 0); // le fond se décale de x pixels  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



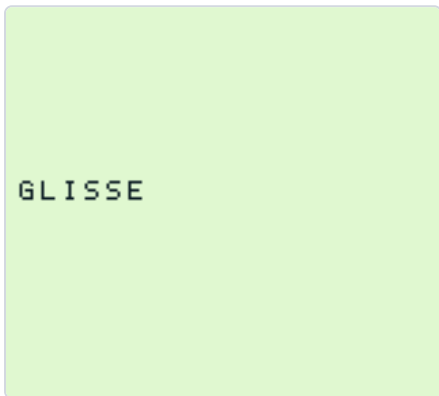
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Le mot GLISSE glisse vers la gauche, puis s'arrête.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le fond est décalé (17 puis 27)
- ✓ et il avance d'un pixel par image (+10)

Ensuite : **0.90.4. L'écran titre — tout l'écran glisse** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++